

01-MyBatis深入剖析

1. 源码灵魂三问

1.1 什么是阅读源码？

- 阅读源码指的是：1.优雅的业务代码、2.经典算法代码、3.优秀框架核心代码

1.2 为什么看源码？

- 阅读优秀源码提升自己代码审美能力的必由之路，阅读源码好似品读经典
- 读源码的程序员不一定是好程序员，不读源码的程序员一定不是好程序员
- 具备分析源码的能力之后，可以大幅降低与百度和Google的耦合度，增加独立解决问题能力，降低解决问题时间
- 面试的大招之一，体现自己的技术掌握的深度

举个栗子：简历写精通MyBatis VS 自定义过MyBatis插件（统计所有查询SQL语句，输出高频查询SQL及耗时）

1.3 如何阅读源码？

高效四小步

- 第一：明确目的
 - 第二：找到地图
 - 第三：梳理主线
 - 第四：做好笔记
-
- 误区："只见树木不见森林"，"贪多嚼不烂"，违背常识阅读源码
 - 常识：源码历史久远、复杂度高、体量大，不可能看完全部细节
 - 建议：反复品味，贪多无益，长读长新
 - 带着目标和地图去读效率最高，有人讲最爽
 - 时间有限，每个框架搞透几个点核心点即可应付面试了

1.4 阅读MyBatis源码目标

- 目标01-掌握SQL语句执行过程
 - 如何参数类型转换及设置参数？
 - 结果集是如何类型转换以及结果集映射的？
- 目标02-掌握mapper接口动态代理原理
- 目标03-理解#{ }与\${ }的本质区别

- 目标04-掌握缓存实现原理
- 目标05-掌握动态SQL实现原理

实战案例：可以手写MyBatis分页插件

2. MyBatis核心内容

2.1 什么是JDBC?

JDBC (Java Database Connectivity) 是Java语言中提供的访问关系型数据的接口。在Java编写的应用中，使用JDBC API可以执行SQL语句、检索SQL执行结果以及将数据更改写回到底层数据源。JDBC API也可以用于分布式、异构的环境中与多个数据源交互。

使用JDBC操作数据源大致的步骤：

- (1) 与数据源建立连接。
- (2) 执行SQL语句。
- (3) 检索SQL执行结果。
- (4) 关闭连接。

JDBC 4.2规范文档：https://download.oracle.com/otndocs/jcp/jdbc-4_2-mrel2-spec/index.html

基于JDBC操作数据库代码，根据username查询用户列表

实现：

```
1 public class JDBCExample {
2     public static void main(String[] args) throws ClassNotFoundException,
        SQLException {
3         //数据库连接地址
4         String url = "jdbc:mysql://localhost:3306/hello";
5         String user = "root";//用户名
6         String password = "root";//密码
7         //1.注册数据库驱动
8         Class.forName("com.mysql.jdbc.Driver");
9         //2.获取数据库连接对象Connection。
10        Connection conn = DriverManager.getConnection(url, user, password);
11        //3.创建Sql语句对象Statement，填写SQL语句
12        PreparedStatement preparedStatement = conn.prepareStatement("select
        * from t_user where name=?");
13        //传入查询参数
14        preparedStatement.setString(1,"刘备【1】");
15        //4.执行SQL查询，返回结果集对象ResultSet
16        ResultSet resultSet = preparedStatement.executeQuery();
17        List<User> users = new ArrayList<User>();
18        //5.循环解析结果集，获取查询用户list集合
19        while (resultSet.next()){
20            User u = User.builder().id(resultSet.getInt("id"))
21                .name(resultSet.getString("name"))
22                .age(resultSet.getInt("age"))
```

```

23         .address(resultSet.getString("address")).build();
24         users.add(u);
25     }
26     //打印查询结果
27     System.out.println(users);
28     //6.关闭连接，释放资源
29     resultSet.close();//关闭结果集对象
30     preparedStatement.close();//关闭Sql语句对象
31     conn.close();//关闭数据库连接对象
32 }
33 }
34

```

JDBC中代码问题分析：

1. 结果集解析复杂，列名硬编码，sql变化导致解析代码变化，系统不易维护。
2. Sql语句硬编码，难以维护。数据库配置硬编码。
3. 频繁连接、释放数据库资源，降低系统性能。
4. preparedStatement向占位符号传参数存在硬编码，不易维护。

2.2 什么是MyBatis?

MyBatis 是一款优秀的持久层框架，它支持**自定义 SQL**、存储过程以及**高级映射**。MyBatis对JDBC的封装，使得**开发者只需要关注SQL语句与业务本身**即可，无需开发者处理加载驱动、获取连接、创建Statement等繁琐的过程。MyBatis 可以通过简单的 XML 或注解来配置和映射原始类型、接口和 Java POJO为数据库中的记录。

MyBatis是一个实现了ORM思想的持久层框架。

什么是ORM? 为什么说MyBatis是一个半ORM框架?

- ORM: Object/Relation Mapping 对象/关系映射。ORM思想：将数据库中的关系数据表映射为JAVA中的对象，把对数据表的操作转换为操作对象，实现面向对象编程。因此，ORM的目的是**使得开发人员以面向对象的思想来操作数据库**。
- MyBatis框架是一个**半自动的ORM**持久层框架，也可以在Java中实现类似 insert(User)的操作最终操作数据库，但是需要我们自己写SQL语句。

2.3 MyBatis基础案例

目标：使用MyBatis框架完成，查询用户列表操作

分析：

1. 导入依赖
2. 配置文件
3. 编写案例代码
 1. 基于配置文件构建SqlSessionFactory工厂对象
 2. 从工厂对象中获取SqlSession
 3. SqlSession执行SQL语句，返回结果

4. 打印

实现:

1. 导入坐标

```
1 <dependencies>
2     <!--导入Mybatis核心jar包-->
3     <dependency>
4         <groupId>org.mybatis</groupId>
5         <artifactId>mybatis</artifactId>
6         <version>3.4.5</version>
7     </dependency>
8     <!--mysql 数据库驱动-->
9     <dependency>
10        <groupId>mysql</groupId>
11        <artifactId>mysql-connector-java</artifactId>
12        <version>5.1.36</version>
13    </dependency>
14    <!--单元测试jar包-->
15    <dependency>
16        <groupId>junit</groupId>
17        <artifactId>junit</artifactId>
18        <version>4.11</version>
19    </dependency>
20    <!--log4j日志jar包-->
21    <dependency>
22        <groupId>log4j</groupId>
23        <artifactId>log4j</artifactId>
24        <version>1.2.17</version>
25    </dependency>
26 </dependencies>
```

2. 编写UserMapper.xml配置文件

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3     PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5 <!--
6 mybatis-3-mapper.dtd 头部的文档声明，验证当前文档的正确性
7 namespace属性：指定当前映射器的命名空间
8 -->
9 <mapper namespace="test">
10     <select id="findAll" resultType="com.hero.pojo.User">
11         select * from user;
12     </select>
13 </mapper>
```

3. 编写SqlMapConfig.xml配置文件

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE configuration
3     PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-config.dtd">
```

```

5 <!--
6 mybatis-3-config.dtd 头部的文档声明，验证当前XML文档的正确性
7 -->
8 <configuration>
9     <environments default="development">
10         <environment id="development">
11             <transactionManager type="JDBC"/>
12             <dataSource type="POOLED">
13                 <property name="driver" value="com.mysql.jdbc.Driver"/>
14                 <property name="url"
15 value="jdbc:mysql://localhost:3306/hello"/>
16                 <property name="username" value="root"/>
17                 <property name="password" value="root"/>
18             </dataSource>
19         </environment>
20     </environments>
21     <!--映射器-->
22     <mappers>
23         <mapper resource="UserMapper.xml"/>
24     </mappers>
25 </configuration>

```

4. 编写MyBatis程序

```

1 public class MybatisExample {
2     @Test
3     public void test() throws IOException {
4         //1.创建SqlSessionFactoryBuilder对象
5         SqlSessionFactoryBuilder builder = new
6 SqlSessionFactoryBuilder();
7         //2.Builder对象构建工厂对象
8         //Resource工具加载配置文件
9         InputStream inputStream =
10 Resources.getResourceAsStream("SqlMapConfig.xml");
11         SqlSessionFactory sqlSessionFactory =
12 builder.build(inputStream);
13         //3.工厂对象Factory打开SqlSession会话
14         SqlSession sqlSession = sqlSessionFactory.openSession();
15         //4.SqlSession会话对象执行SQL语句，selectList(命名空间+查询语句唯一标
16 识)
17         List<User> users = sqlSession.selectList("test.findAll");
18         //5.打印查询结果
19         for (User u : users) {
20             System.out.println(u);
21         }
22         //6.关闭SqlSession会话
23         sqlSession.close();
24     }
25 }

```

2.4 两种开发方式

使用MyBatis开发Dao，通常有两种方法：

方式一：基于Dao层实现类开发

- 需要编写SQL语句，需要编写接口的实现类

方式二：基于Mapper动态代理自动生成Dao层实现类

- 需要编写SQL语句，不需要编写接口的实现类
- 必须遵守的四个规范：
 1. 命名空间要与接口的全限定名相同
 2. SQL标签id，必须与对应接口方法名称相同
 3. SQL标签参数类型，与对应接口方法的返回值类型相同
 4. SQL标签返回值类型，与对应接口方法的参数类型相同

```
1 // 方式1：基于Dao层实现类开发
2 User user = session.selectOne("com.hero.dao.UserMapper.findUserById", 110);
3 // 方式2：基于Mapper动态代理自动生成Dao层实现类
4 UserMapper mapper = session.getMapper(UserMapper.class);
5 List<User> userList = mapper.findAll();
```

2.5 全局配置文件解析

- **全局属性properties**：抽取公共属性配置，增加配置属性代码的可维护性，支持读取外置属性配置文件db.properties
- **注册别名typeAliases**：类型别名存在的意义是减少类全限定名的冗余
- **数据源环境environments**：配置数据库环境信息，与事务管理器
- **配置mappers**：加载SQL语句对应的mapper文件

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE configuration
3     PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
4     "http://mybatis.org/dtd/mybatis-3-config.dtd">
5 <configuration>
6     <!--
7     全局properties属性值
8     resources属性值：设置外置数据库properties全局配置文件路径
9     -->
10    <properties resource="db.properties">
11        <!--
12        配置全局property属性键值key-value
13        name属性：设置键
14        value属性：设置值
15        properties引入的外部配置文件的属性优先被读取。外部properties覆盖内部全局配置
16        -->
17        <property name="jdbc.driver" value="com.mysql.jdbc.Driver"/>
18        <property name="jdbc.url" value="jdbc:mysql://localhost:3306/test03?
19        characterEncoding=utf8"/>
20        <property name="jdbc.username" value="root"/>
21        <property name="jdbc.password" value="root"/>
```

```

21     </properties>
22
23     <environments default="development">
24         <environment id="development">
25             <transactionManager type="JDBC"/>
26             <dataSource type="POOLED">
27                 <property name="driver" value="${jdbc.driver}"/>
28                 <property name="url" value="${jdbc.url}"/>
29                 <property name="username" value="${jdbc.username}"/>
30                 <property name="password" value="${jdbc.password}"/>
31             </dataSource>
32         </environment>
33     </environments>
34     <mappers>
35         <mapper resource="UserMapper.xml"/>
36     </mappers>
37 </configuration>

```

2.6 参数类型与结果映射

- 参数类型parameterType
- 返回值类型resultType
- 返回值类型自定义映射resultMap，比自动映射resultType更加灵活

```

1  <!--
2      parameterType: 1.基本数据类型及其包装类, 2.字符串, 3.pojo实体类, 5.类型别名,
3      4.pojo实体类包装类
4      #{} , 取值可以通过点的形式取出对象中对象的成员变量值
5      resultType: 1.基本数据类型及其包装类, 2.字符串, 3.pojo实体类, 5.类型别名, 4.pojo
6      实体类包装类
7      -->
8  <select id="findByUsername" parameterType="QueryVo" resultType="user">
9      select * from user where username=#{user.username};
10 </select>
11 <select id="findAll" resultMap="findAllResultMap">
12     select * from `order`;
13 </select>
14 <!--自定义查询结果集映射: id是唯一标识、type设置resultMap结果类型, 可以是别名, 也可以是
15 类全限定名-->
16 <resultMap id="findAllResultMap" type="order">
17     <!-- 映射主键: property指定实体类变量、column指定列名-->
18     <id property="id" column="id"/>
19     <!-- 映射对象普通成员变量 -->
20     <result property="userId" column="user_id"/>
21     <result property="number" column="number"/>
22     <result property="createtime" column="createtime"/>
23     <result property="note" column="note"/>
24 </resultMap>

```

#{ }与\${ }区别:

#{ }:

- `#{}` 等同于PreparedStatement中的占位符 `?`，会自动对传入的字符串数据加一对单引号，可以避免SQL注入
- `#{}` 可以接收简单类型值或Pojo属性值
- 如果parameterType传输单个简单类型值，`#{}` 括号中可以是任意名称

`${}` :

- `${}` 将传入的数据直接显示生成在SQL中，只是简单的SQL字符串拼接。
- `${}` 可以接收简单类型值或Pojo属性值
- 如果parameterType传输单个简单类型值，`${}` 括号中只能是“value”这个字符串。
 - 注意：这个特点仅限于3.4.5版本，3.5.2版本以后与`#{}` 就与`#{}` 一样了，也可以是任意名称

注意：一般能用#的就别用\$

问题：`#{}` 与`${}` 区别的本质是什么？

2.7 动态SQL语句

动态SQL就是SQL语句的动态拼接(生成)。

常用标签有5个：`if`标签、`where`标签、`set`标签、`sql`标签、`foreach`标签

- `if`标签作用：根据条件拼接SQL语句
- `where`标签作用：自动生成`where`，去除多余`and`关键词
- `set`标签作用：自动生成`update`语句的`set`，去除多余逗号分隔符
- `sql`标签作用：抽取公用SQL语句
 - `include`标签作用：引入公共SQL语句标签的SQL
- `foreach`标签作用：遍历查询list集合参数，拼接SQL语句

```

1  <!-- where与if标签 -->
2  <select id="findUserByWhere" parameterType="user" resultType="user">
3      select * from user
4      <!-- 自动生成where，去除and关键词 -->
5      <where>
6          <!-- 根据条件拼接SQL语句，test是判断条件：为true时执行，false不执行 -->
7          <if test="username!=null and username!=''">
8              and username like #{username}
9          </if>
10         <if test="sex!=null and sex !='">
11             and sex=#{sex};
12         </if>
13     </where>
14 </select>
15 <!-- set标签 -->
16 <update id="update" parameterType="user">
17     update user
18     <!-- 自动生成set，去除修改条件后的逗号分隔符 -->
19     <set>
20         <if test="username!=null and username!=''">
21             username = #{username},

```



```

22         </if>
23         <if test="sex!=null and sex!=''">
24             sex=#{sex},
25         </if>
26     </set>
27     where id=#{id};
28 </update>
29 <!-- sql标签: 抽取公共SQL语句-->
30 <sql id="usersql">
31     id, username, birthday, sex, address
32 </sql>
33 <select id="findById" parameterType="int" resultType="user">
34     <!-- 引入公共SQL语句, refid属性值: SQL标签的id -->
35     select <include refid="usersql"/> from user where id=#{id};
36 </select>
37 <!-- foreach标签-->
38 <select id="findUserByIds" parameterType="list" resultType="user">
39     select <include refid="usersql"/> from user where id in
40     <!-- 遍历查询list集合参数, 拼接SQL语句
41         1.collection: 指定集合, 如果是List集合写list, 是普通数组写array
42         2.item: 每个元素的变量, 类似于for循环的i变量
43         3.separator: 元素之间的分隔符
44         4.open: 开始部分SQL语句
45         5.close: 结束部分SQL语句 -->
46     <foreach collection="list" item="id" separator="," open="(" close=")">
47         #{id}
48     </foreach>
49 </select>

```

问题：动态SQL语句的实现原理是什么？

2.8 多表关联查询

多表关联：至少两个表关联。分析多表关系的经验技巧：**从一条记录出发，不要从表整体去分析**，比如分析A表和B表关系，A表中的一条记录对应B表中的几条记录，如果是一条，那么A到B就是一对一；如果多条，那么A到B就是一对多

- 一对一，从账户表出发，到用户表，是一对一的关系
- 一对多，从用户表出发，到账户表，一条用户数据可以在账户表中存在多条记录，这就是一对多
- 多对多，用户和角色表，一个用户可以有很多角色，一个角色有很多用户，多对多通过中间表来体现

```

1 <!-- 一对一关系: Account与User-->
2 <resultMap id="findAllResultMap" type="account">
3     <id property="id" column="id"/>
4     <result property="uid" column="uid"/>
5     <result property="money" column="money"/>
6     <!-- 一对一映射: 映射account对象内user对象的查询结果
7         property: 定义account对象的成员变量(user)
8         javaType: 定义映射的成员变量的数据类型 -->
9     <association property="user" javaType="user">
10         <id property="id" column="uid"/>
11         <result property="username" column="username"/>
12         <result property="address" column="address"/>
13     </association>

```

```

14 </resultMap>
15 <select id="findAll" resultMap="findAllResultMap">
16     select * from account left join user u on account.uid = u.id;
17 </select>
18
19 <!-- 一对多关系: User与Account -->
20 <resultMap id="findAllResultMap" type="user">
21     <id property="id" column="id"/>
22     <result property="username" column="username"/>
23     <result property="birthday" column="birthday"/>
24     <result property="sex" column="sex"/>
25     <result property="address" column="address"/>
26     <!-- 一对多映射: 映射user对象中的账户信息list集合数据
27         property: 定义user对象中的集合类型的成员变量, 映射user对象的accounts是
list集合
28         ofType: 定义list集合内元素的数据类型 -->
29     <collection property="accounts" ofType="account">
30         <id property="id" column="aid"/>
31         <result property="uid" column="uid"/>
32         <result property="money" column="money"/>
33     </collection>
34 </resultMap>
35 <select id="findAll" resultMap="findAllResultMap">
36     select user.*,a.id aid,a.uid,a.money from user left join account a on
user.id = a.uid;
37 </select>
38
39 <!-- 多对多关系: Role和用户 -->
40 <resultMap id="findAllResultMap" type="role">
41     <id property="roleId" column="r_id"/>
42     <result property="roleName" column="r_name"/>
43     <result property="roleDesc" column="r_desc"/>
44     <!-- 多对多映射: 多对多其实是三张表的双向一对多, 也可以用collection标签来配置-->
45     <!-- 映射role对象中的成员变量users, list集合-->
46     <collection property="users" ofType="user">
47         <id property="id" column="id"/>
48         <result property="username" column="username"/>
49         <result property="sex" column="sex"/>
50         <result property="birthday" column="birthday"/>
51         <result property="address" column="address"/>
52     </collection>
53 </resultMap>
54 <!-- 查询所有角色信息, 并关联查询出该角色下的所有用户信息 -->
55 <select id="findAll" resultMap="findAllResultMap">
56     select
57         r.r_id,r.r_name,r.r_desc,u.id,u.username,u.sex,u.birthday,u.address
58     from role r
59         left join user_role ur on r.r_id=ur.rid
60         left join user u on ur.uid=u.id;
61 </select>

```

2.9 延迟加载

Mybatis的一对一，一对多，多对多关系的配置可以实现对象的关联查询。实际开发过程中很多时候我们并不需要总是在加载用户信息时就一定要加载他的账户信息，这种需求可以通过懒加载来实现。

延迟加载分为：一对一延迟加载、一对多延迟加载、全局延迟加载

- 延迟加载针对的是关联对象
- 想要延迟加载用户的数据，关联sql语句肯定要拆分！一条联表查询的SQL语句变为两条SQL。

```
1  <!-- 一对一延迟加载 -->
2  <resultMap id="findAllResultMap" type="account">
3      <id property="id" column="id"/>
4      <result property="uid" column="uid"/>
5      <result property="money" column="money"/>
6      <!-- 配置懒加载
7          select: 设置一对一关联查询执行的SQL语句的唯一标识
8          column: 设置调用查询SQL语句传入的参数
9          fetchType: 设置延迟加载策略，懒加载lazy和立即加载eager -->
10     <association property="user" javaType="user" fetchType="lazy"
11         select="findById" column="uid"/>
12 </resultMap>
13 <select id="findAll" resultMap="findAllResultMap">
14     select * from account;
15 </select>
16 <select id="findById" resultType="user" parameterType="int">
17     select * from user where id=#{id};
18 </select>
19 <!-- 一对多延迟加载 -->
20 <resultMap id="findAllResultMap" type="user">
21     <id property="id" column="id"/>
22     <result property="username" column="username"/>
23     <result property="birthday" column="birthday"/>
24     <result property="sex" column="sex"/>
25     <result property="address" column="address"/>
26     <!--
27         映射一对多查询结果信息
28         select: 设置一对多，关联查询的SQL语句的唯一标识
29         column: 设置关联查询SQL语句传入的参数
30         ofType: 设置集合中元素的数据类型
31         fetchType: 开启拦截，lazy懒加载，eager立即加载
32     -->
33     <collection property="accountList" select="findAccountsByUserId"
34         ofType="account" column="id" fetchType="eager"/>
35 </resultMap>
36 <select id="findAll" resultMap="findAllResultMap">
37     select id, username, birthday, sex, address from user;
38 </select>
39 <select id="findAccountsByUserId" parameterType="int" resultType="account">
40     select * from account where uid=#{userid};
41 </select>
```

配置全局懒加载

```

1 <!-- 全局mybatis配置 -->
2 <settings>
3     <!-- 设置具体配置值: key-value
4         lazyLoadingEnabled: 开启全局延迟加载 -->
5     <setting name="lazyLoadingEnabled" value="true"/>
6 </settings>

```

2.10 缓存

像多数持久化框架一样，Mybatis也提供缓存策略，通过缓存策略来减少数据库的查询次数，从而提高性能。

Mybatis缓存分为：

- 一级缓存：默认开启，SqlSession级别的缓存，只要SqlSession没有commit、flush或close，它就存在
- 二级缓存：需要手动开启，mappers级别的缓存，多个SqlSession去操作同一个Mapper映射的sql语句，多个SqlSession可以共用二级缓存，二级缓存是跨SqlSession的

```

1 <!-- 开启二级缓存配置 -->
2 <!-- 全局开启 -->
3 <settings>
4     <!-- cacheEnabled 全局缓存配置，默认值为true，开启状态 -->
5     <setting name="cacheEnabled" value="true"/>
6 </settings>
7 <!-- 局部mapper级别开启 -->
8 <cache/>
9 <!-- 针对某个select级别开启
10     useCache: 设置select查询语句使用二级缓存，默认值为true-->
11 <select id="findById" parameterType="int" resultType="user" useCache="true">
12     select * from user where id=#{id};
13 </select>

```

问题：缓存的原理是什么？缓存的生命周期是啥？

2.11 注解开发

Mybatis最大的特点是使用XML存储SQL语句，但是Mybatis也可以使用注解开发方式，好处是不用编写Mapper映射文件，坏处是SQL语句又耦合在Java代码中了。通过注解方式，就不再需要去编写UserMapper.xml映射文件了。

Mybatis的注解说明：

- @Insert实现新增，作用与Insert标签相同
- @Update实现更新，作用与Update标签相同
- @Delete实现删除，作用与Delete标签相同
- @Select实现查询，作用与Select标签相同
- @Result实现结果集封装
- @Results可以与@Result一起使用，封装多个结果集
- @One实现一对一结果集封装

- @Many实现一对多结果集封装
- @SelectProvider:实现动态SQL映射，作用与insert标签相同

```

1  public interface UserMapper {
2      //查询
3      @Select("select * from user where id=#{id};")
4      User findById(Integer id);
5      //新增
6      @Insert("insert into user (username, birthday) VALUES (#{username},#{
7      {birthday}})")
8      int save(User user);
9      //修改
10     @Update("update user set username=#{username},address=#{address} where
11     id=#{id}")
12     int update(User user);
13     //删除
14     @Delete("delete from user where id=#{id}")
15     void delete(Integer id);
16     //一对多
17     @Select("select * from user;")
18     @Results({
19         @Result(id = true,property = "id",column = "id"),
20         @Result(property = "username",column = "username"),
21         @Result(property = "birthday",column = "birthday"),
22         @Result(property = "sex",column = "sex"),
23         @Result(property = "address",column = "address"),
24         @Result(property = "accountList",column = "id",
25             javaType = List.class,
26             many = @Many(select =
27             "com.hero.mapper.AccountMapper.findAccountByUserId",fetchType =
28             FetchType.LAZY))
29     })
30     List<User> findAll();
31 }
32 public interface AccountMapper {
33     //一对一
34     @Select("select * from account;")
35     @Results({
36         @Result(id = true,property = "id",column = "id"),
37         @Result(property = "uid",column = "uid"),
38         @Result(property = "money",column = "money"),
39         @Result(property = "user",column = "uid",
40             javaType = User.class,
41             one = @One(select =
42             "com.hero.mapper.UserMapper.findById",fetchType = FetchType.LAZY))
43     })
44     List<Account> findAll();
45 }

```

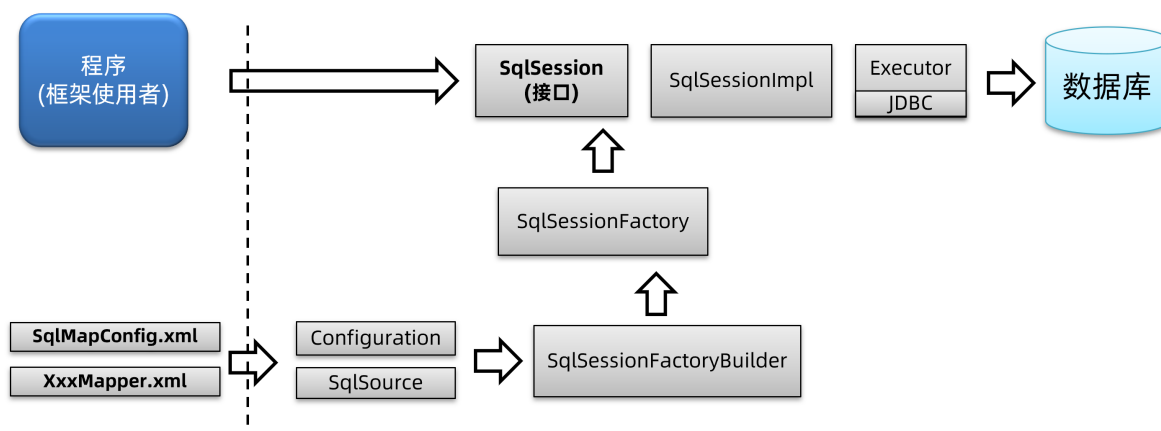
问题：注解可以使用动态SQL语句的特性吗？如何使用

3. 案例：手写MyBatis框架

目标：基于已经学过的知识，设计一个持久层框架。

分析：

- 目的
 - 简化JDBC持久层开发的代码
 - 感受框架的产生过程
 - 通过这个过程可以帮助我们了解底层
- 角色
 - 框架使用者（码农）
 - 框架设计者（攻城狮）
- 注意
 - 类的设计遵循**单一职责原则**



步骤：

1. 自定义框架准备工作，SQL配置文件及其封装类，数据库信息配置文件及其封装类
2. 编写SqlSession接口类和实现类：负责提供常用的数据库操作API接口
3. 编写SqlSessionFactory类：负责创建SqlSession对象的实现类
4. 编写SqlSessionFactoryBuilder类-解析核心配置文件
5. 编写SqlSessionFactoryBuilder类-解析Sql配置文件
6. 安装自定义框架到Maven仓库

3.1 实现

第一步：自定义框架准备工作

1.1 创建Maven工程，导入依赖坐标：

```

1  <!--dom for java 读取xml文件-->
2  <dependency>
3      <groupId>dom4j</groupId>
4      <artifactId>dom4j</artifactId>
5      <version>1.6.1</version>
6  </dependency>
7  <!--xpath表达式，快速定位xml文件元素-->
8  <dependency>
9      <groupId>jaxen</groupId>
10     <artifactId>jaxen</artifactId>
11     <version>1.1.6</version>
12 </dependency>

```

1.2 配置文件结构约定：数据库配置信息，及Sql配置文件

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <configuration>
3      <dataSource>
4          <property name="driver" value="com.mysql.jdbc.Driver" />
5          <property name="url" value="jdbc:mysql://127.0.0.1:3306/hello?
characterEncoding=utf8" />
6          <property name="username" value="root" />
7          <property name="password" value="root" />
8      </dataSource>
9      <mappers>
10         <mapper resource="UserMapper.xml"/>
11     </mappers>
12 </configuration>

```

```

1  <?xml version="1.0" encoding="utf-8" ?>
2  <mapper namespace="com.hero.dao.UserMapper">
3      <select id="queryUserList" resultType="com.hero.pojo.com.User">
4          select * from user
5      </select>
6  </mapper>

```

1.3 将数据库配置文件和Sql配置文件封到类中

```

1  public class Configuration {
2      //数据库驱动
3      private String driver;
4      //数据库连接地址
5      private String url;
6      //用户名
7      private String username;
8      //密码
9      private String password;
10     //存储Mapper对象，Mapper对象封装Sql语句和结果集的JavaBean
11     //Mapper可能有多，所以存储在Mapper集合中
12     //Map集合的key是，namespace属性值++.id属性值
13     //Map集合的value是Mapper对象
14     private Map<String,Mapper> map = new HashMap<String, Mapper>();

```

```

15     //getter() setter() toString()
16 }

```

```

1 public class SqlSource {
2     //存储了Sql语句
3     private String sql;
4     //存储封装JavaBean对象类的全限定名称
5     private String resultType;
6     //getter() setter() toString()
7 }

```

第二步：编写SqlSession接口及其实现类

```

1 /**
2  * 对外暴露SqlSession接口，接口中定义常见的JDBC操作：增删改查
3  */
4 public interface SqlSession {
5     /**
6      * 查询所有用户
7      * T 代表泛型类型，T(type缩写)
8      */
9     <T> List<T> selectList(String sql) throws Exception;
10 }

```

```

1 public class SqlSessionImpl implements SqlSession {
2
3     //每次sql会话连接，必须要有数据库配置信息
4     private Configuration configuration;
5
6     public SqlSessionImpl(Configuration configuration) {
7         this.configuration = configuration;
8     }
9
10    public <T> List<T> selectList(String statement) throws Exception{
11        Executor executor = new Executor(configuration);
12
13        return (List<T>) executor.executeQuery(statement);
14    }
15 }

```

简化实现类代码，抽取公用代码到工具类

```

1 public class Executor {
2     private Configuration configuration;
3
4     public Executor(Configuration configuration) {
5         this.configuration = configuration;
6     }
7
8     public List executeQuery(String statement) throws Exception {
9         /**

```



```

10      * 数据库连接信息硬编码
11      */
12      String driver = configuration.getDriver();
13      String url = configuration.getUrl();
14      String username = configuration.getUsername();
15      String password = configuration.getPassword();
16      Map<String, SqlSource> map = configuration.getSqlSourceMap();
17      //SQL映射对象
18      SqlSource mapper = map.get(statement);
19      String sqlStr = mapper.getSql(); //获取查询SQL
20      String resultType = mapper.getResultType(); //获取返回值类的全限定名(包名
+类名)
21
22      //1.注册MySQL驱动
23      Class.forName(driver);
24      //2.获取连接Connection对象
25      /**
26       * 数据库连接的创建和释放频繁,造成系统资源的浪费,严重影响了系统性能
27       */
28      Connection conn = DriverManager.getConnection(url, username,
password);
29      //3.创建SQL语句对象Statement,填写SQL语句
30      PreparedStatement ps = conn.prepareStatement(sqlStr);
31      //4.执行查询SQL,返回结果集ResultSet
32      ResultSet rs = ps.executeQuery();
33      //5.解析结果集,获取查询用户list集合
34      //获取结果集元数据
35      ResultSetMetaData metaData = rs.getMetaData();
36      //获取总列数5
37      int columnCount = metaData.getColumnCount();
38      //获取所有列的list集合
39      List<String> columnNames = new ArrayList<String>();
40      for (int i = 1; i <= columnCount; i++) {
41          columnNames.add(metaData.getColumnName(i));
42      }
43      List list = new ArrayList();
44      //循环解析结果集
45
46      while (rs.next()) {
47          //通过反射获取类的字节码对象,传入的参数是类的全限定名称
48          Class<?> clazz = Class.forName(resultType);
49          //反射创建对象
50          Object user = clazz.newInstance();
51          //反射获取当前类的所有方法
52          Method[] methods = clazz.getMethods();
53          //循环 遍历所有列名
54          for (String columnName : columnNames) {
55
56              for (Method method : methods) {
57                  String methodName = method.getName();
58                  //判断方法的名称,与set+列名相等,那么就把列名对应的值设置到当前对
象的set方法中
59                  if (("set"+columnName).equalsIgnoreCase(methodName)){
60                      //把列名column对应的值,设置到对象的set方法中,给属性赋值
61                      method.invoke(user, rs.getObject(columnName));

```

```

62         }
63     }
64 }
65     //将用户存入集合中
66     list.add(user);
67 }
68     //关闭连接，释放资源
69     rs.close();
70     ps.close();
71     conn.close();
72     return list;
73 }
74 }

```

第三步：编写SqlSessionFactory类

```

1  /**
2   * SqlSession工厂类，负责创建SqlSession接口实现类
3   */
4  public class SqlSessionFactory {
5
6      private Configuration configuration;
7
8      public SqlSessionFactory(Configuration configuration) {
9          this.configuration = configuration;
10     }
11
12     /**
13      * 创建SqlSession会话
14      */
15     public SqlSession openSession(){
16         return new SqlSessionImpl(configuration);
17     }
18 }

```

第四步：编写SqlSessionFactoryBuilder类

解析数据库配置文件

```

1  /**
2   * 工厂对象构建器对象，负责构建工厂对象，解析配置文件
3   */
4  public class SqlSessionFactoryBuilder {
5
6      /**
7       * 构建工厂对象
8       * 参数: SqlMapConfig.xml配置文件的输入流对象
9       */
10     public SqlSessionFactory build(InputStream inputStream) throws
11     DocumentException {
12         Configuration configuration = new Configuration();
13         //解析配置文件
14         loadXmlConfig(configuration,inputStream);
15         return new SqlSessionFactory(configuration);
16     }
17 }

```

```

15     }
16
17     /**
18      * 解析框架使用者传入的配置文件
19      */
20     private void loadXmlConfig(Configuration configuration,InputStream
inputStream) throws DocumentException {
21         //创建解析XML文件对象SAXReader
22         SAXReader saxReader = new SAXReader();
23         //读取SqlMapConfig.xml配置文件流资源，获取文档对象
24         Document document = saxReader.read(inputStream);
25         //获取SqlMapConfig.xml 配置文件内所有property标签元素
26         List<Element> selectNodes = document.selectNodes("//property");
27         //循环解析property标签内容，抽取配置信息
28         for (Element element : selectNodes) {
29             String name = element.attributeValue("name");
30             if ("driver".equals(name)){//数据库驱动
31                 configuration.setDriver(element.attributeValue("value"));
32             } else if ("url".equals(name)){//数据库地址
33                 configuration.setUrl(element.attributeValue("value"));
34             } else if ("username".equals(name)){//用户名
35                 configuration.setUsername(element.attributeValue("value"));
36             } else if ("password".equals(name)){//密码
37                 configuration.setPassword(element.attributeValue("value"));
38             }
39         }
40         //解析SqlMapConfig.xml 映射器配置信息
41         List<Element> list = document.selectNodes("//mapper");
42         for (Element element : list) {
43             //SQL映射配置文件路径
44             String resource = element.attributeValue("resource");
45             //解析SQL映射配置文件
46             loadSqlConfig(resource,configuration);
47         }
48     }
49 }

```

第五步：解析SQL配置文件

```

1     /**
2      * 解析SQL配置文件
3      */
4     private void loadSqlConfig(String resource, Configuration configuration)
throws DocumentException {
5         //根据SQL映射配置文件路径，读取流资源。classpath路径下
6         InputStream inputStream =
this.getClass().getClassLoader().getResourceAsStream(resource);
7         //创建解析XML文件的对象SAXReader
8         SAXReader saxReader = new SAXReader();
9         //读取UserMapper.xml配置文件文档对象
10        Document document = saxReader.read(inputStream);
11        //获取文档对象根节点: <mapper namespace="test">
12        Element rootElement = document.getRootElement();
13        //取出根节点的命名空间
14        String namespace = rootElement.attributeValue("namespace");

```

```

15 //获取当前SQL映射文件所有查询语句标签
16 List<Element> selectNodes = document.selectNodes("//select");
17 //循环解析查询标签select, 抽取SQL语句
18 for (Element element : selectNodes) {
19     //查询语句唯一标识
20     String id = element.attributeValue("id");
21     //当前查询语句返回结果集对象类型
22     String resultType = element.attributeValue("resultType");
23     //查询语句
24     String sql = element.getText();
25     //创建Mapper对象
26     SqlSource mapper = new SqlSource();
27     mapper.setSql(sql);
28     mapper.setResultType(resultType);
29     //在configuration中设置mapper类, key: (命名空间++.SQL语句唯一标识符)
30     configuration.getSqlSourceMap().put(namespace+"."+id,mapper);
31 }
32 }

```

最后：自定义框架安装到Maven仓库

3.2 测试框架

测试自定义的MyBatis框架

步骤：

1. 导入自定义框架的坐标
2. 定义User实体类
3. 配置文件：数据库配置文件，和SQL配置文件
4. 编写测试代码

实现过程：

1. 创建Maven工程，导入自定义框架的坐标

```

1 <!--自定义框架坐标jar包-->
2 <dependency>
3     <groupId>com.hero</groupId>
4     <artifactId>mybatis-custom-frame</artifactId>
5     <version>1.0-SNAPSHOT</version>
6 </dependency>
7 <!--单元测试jar包-->
8 <dependency>
9     <groupId>junit</groupId>
10    <artifactId>junit</artifactId>
11    <version>4.12</version>
12 </dependency>
13 <!--mysql数据库驱动-->
14 <dependency>
15     <groupId>mysql</groupId>
16     <artifactId>mysql-connector-java</artifactId>
17     <version>5.1.36</version>
18 </dependency>

```

```

19 <dependency>
20     <groupId>org.projectlombok</groupId>
21     <artifactId>lombok</artifactId>
22     <version>1.16.18</version>
23     <scope>provided</scope>
24 </dependency>

```

2. 创建实体类(Pojo)

```

1 @Data
2 public class User {
3     private int id;
4     private String username;
5     private String sex;
6     private Date birthday;
7     private String address;
8 }

```

3. 创建配置文件：数据库配置文件，和SQL配置文件

SqlMapConfig.xml

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <configuration>
3     <environments>
4         <environment>
5             <dataSource>
6                 <property name="driver" value="com.mysql.jdbc.Driver"/>
7                 <property name="url"
8 value="jdbc:mysql://localhost:3306/hello?characterEncoding=utf8" />
9                 <property name="username" value="root"/>
10                <property name="password" value="root"/>
11            </dataSource>
12        </environment>
13    </environments>
14    <mappers>
15        <mapper resource="UserMapper.xml"/>
16    </mappers>
17 </configuration>

```

UserMapper.xml

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <mapper namespace="com.hero.dao.UserMapper">
3     <select id="findAll" resultType="com.hero.pojo.User">
4         select * from t_user;
5     </select>
6 </mapper>

```

4. 编写测试代码

```

1 public class CustomFrameTest {
2     @Test
3     public void test() throws Exception {

```

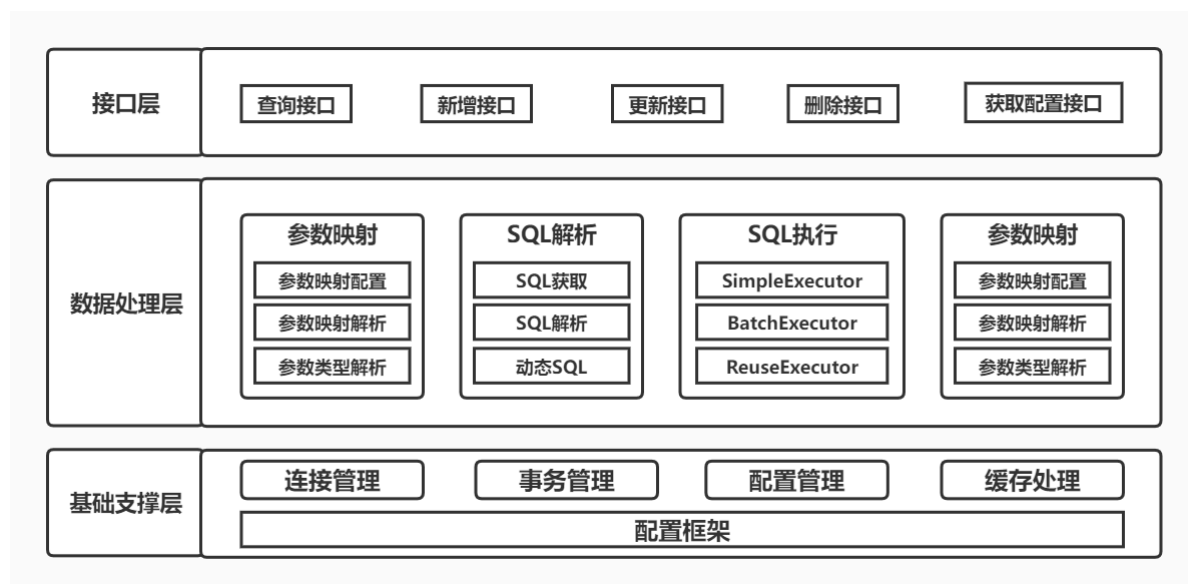
```

4      //1.创建SqlSessionFactoryBuilder对象
5      SqlSessionFactoryBuilder builder = new
SqlSessionFactoryBuilder();
6      //2.builder对象构建工厂对象
7      InputStream inputStream =
this.getClass().getClassLoader().getResourceAsStream("sqlMapConfig.xml")
;
8      SqlSessionFactory sqlSessionFactory =
builder.build(inputStream);
9      //3.打开SqlSession会话
10     SqlSession sqlSession = sqlSessionFactory.openSession();
11     //4.执行查询Sql语句
12     List<User> users =
sqlSession.selectList("com.hero.dao.UserMapper.findAll");
13     //5.循环打印
14     for (User u : users) {
15         system.out.println(u);
16     }
17 }
18 }

```

4. 高级别架构图

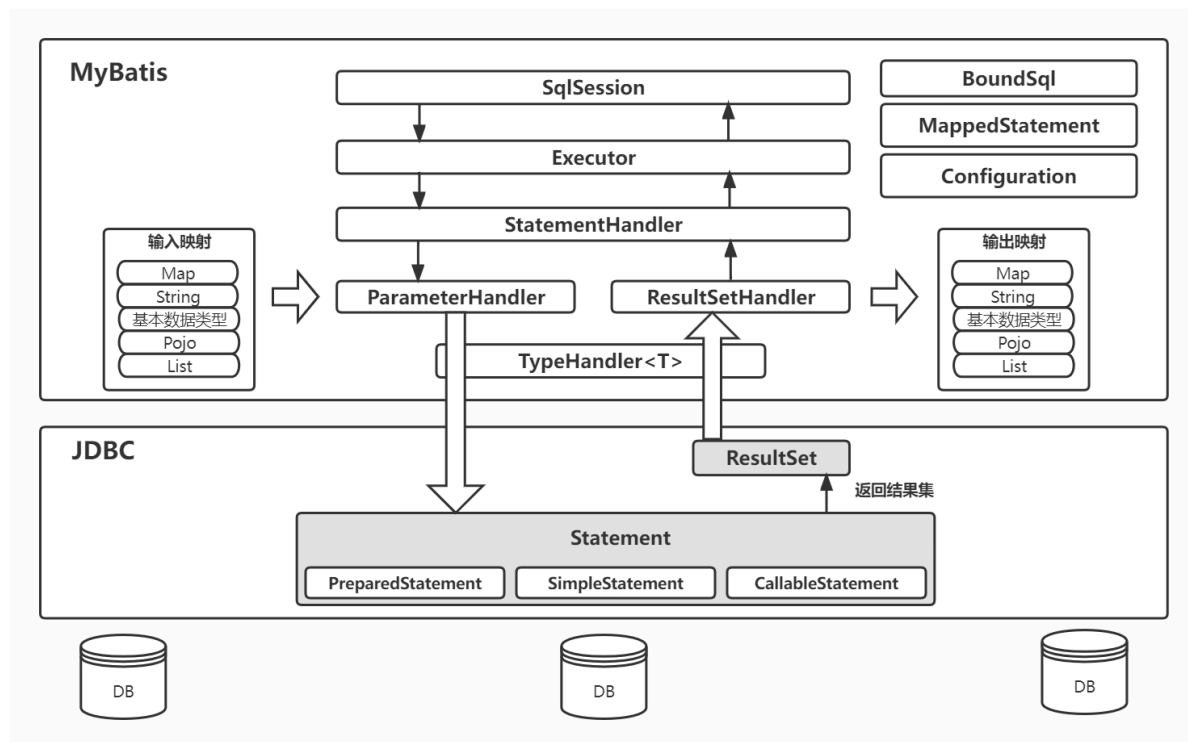
4.1 分层模块架构图



- **接口层：**MyBatis提供给开发人员的一套API，主要使用的是SqlSession接口。
 - 主要负责：为开发者提供统一使用的接口
 - 有两种开发方式：1.基于Dao层接口实现类开发，2.基于动态代理自动生成Dao层接口实现类
- **数据处理层：**MyBatis框架内部的核心实现，主要负责：
 - 参数解析、参数绑定
 - SQL解析，动态拼接SQL
 - 结果集映射解析、映射处理

- **支撑层**：用来完成MyBatis与数据库基本连接方式以及SQL命令与配置文件对应，主要负责：
 - 连接管理
 - 事务管理
 - 配置加载
 - 缓存管理

4.2 核心组件架构图



Mybatis核心组织层次结构：四个组件三个封装类

四个组件

1. **SqlSession**：顶层API接口，对外暴露接口对数据库进行增删改查操作。通过会话工厂对象创建
 - 默认实现类是DefaultSQLSession类
2. **Executor**：执行器，是MyBatis的核心，负责SQL动态语句的生成和查询缓存的维护
3. **StatementHandler**：负责处理JDBC的Statement的交互
 - **ParameterHandler**：负责参数设置
 - **ResultSetHandler**：负责结果集映射
4. **TypeHandler**：负责JdbcType到JavaType之间的数据转换

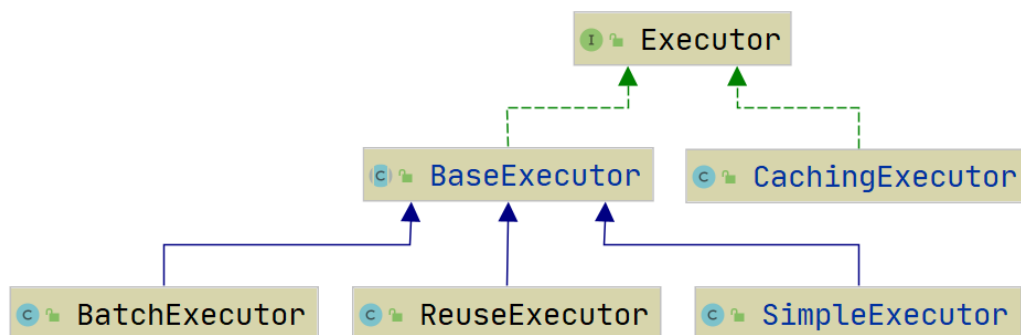
三个封装类

1. **Configuration**：封装全局配置文件和映射文件中的信息
2. **BoundSql**：封装解析之后的SQL语句和解析出来的参数信息
3. **MappedStatement**：封装select|insert|update|delete等标签信息，一个标签对应一个MappedStatement对象，包含入参和出参类型信息
 - Executor通过MappedStatement在执行sql前，将**输入java对象属性映射至SQL中**，输入参数映射就是jdbc编程中对preparedStatement设置参数
 - Executor通过MappedStatement在执行sql后，将**输出结果映射至java对象属性中**，输出结果映射过程相当于jdbc编程中对结果的解析处理过程

4.2.1 其他相关类简介

- **SqlSessionFactoryBuilder**: 构建SqlSessionFactory
- **XMLConfigBuilder**: 专门用来解析全局配置文件的解析器
- **XMLMapperBuilder**: 专门用来解析映射文件的解析器
- **SqlSessionFactory接口**: 默认实现类是DefaultSQLSessionFactory类
- **SqlSource接口**: SqlSource用于描述SQL资源, 与动态SQL、#()和\${}表达式相关
 - **ProviderSqlSource**: 用于描述通过@Select、@SelectProvider等注解配置的SQL资源信息。
 - **DynamicSqlSource**: 用于描述Mapper XML文件中配置的SQL资源信息, 这些SQL通常包含动态SQL配置或者\${}参数占位符, 需要在Mapper调用时才能确定具体的SQL语句。
 - **RawSqlSource**: 用于描述Mapper XML文件中配置的SQL资源信息, 与DynamicSqlSource不同的是, 这些SQL语句在解析XML配置的时候就能确定, 即不包含动态SQL相关配置。
 - **StaticSqlSource**: 用于描述ProviderSqlSource、DynamicSqlSource及RawSqlSource解析后得到的静态SQL资源。

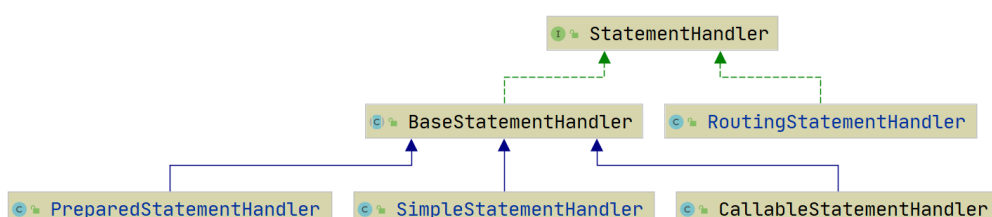
4.2.2 Executor继承体系



- SimpleExecutor简单执行器
- ReuseExecutor可重用预处理器执行器
- BatchExecutor批处理执行器

这三个执行器继承了一个BaseExecutor基础执行器, 而这个基础执行器实现了Executor接口, 其中SimpleExecutor是默认的执行器。CachingExecutor是二级缓存执行器, 当开启二级缓存是则会实例化它, 在BaseExecutor 的基础上, 包装了二级缓存功能!

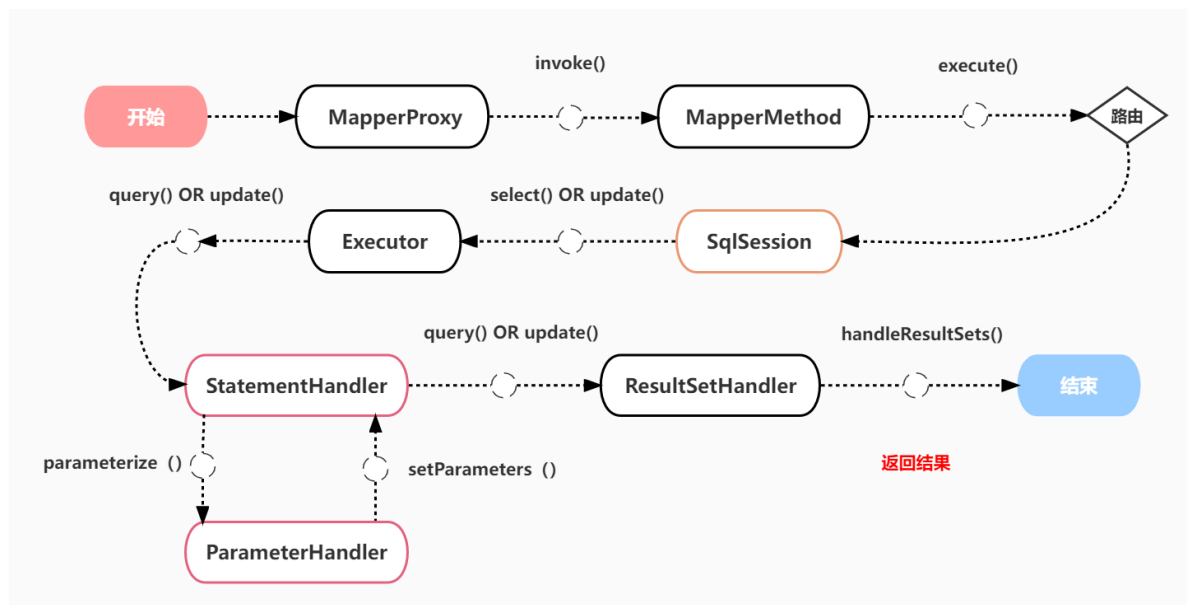
4.2.3 StatementHandler继承体系



- SimpleStatementHandler简单的statement
- CallableStatementHandler存储过程statement
- PreparedStatementHandler预编译的statement
- RoutingStatementHandler路由Statement，默认的StatementHandler，它会根据Executor类型创建对应的StatementHandler，保存到属性delegate中

5. 源码阅读

5.1 SQL执行过程



5.1.1 SQL执行入口

```

1 // 方式1: 基于Dao层实现类开发
2 User user = session.selectOne("com.hero.dao.UserMapper.findUserId", 110);
3
4 // 方式2: 基于Mapper动态代理自动生成Dao层实现类
5 UserMapper mapper = session.getMapper(UserMapper.class);
6 List<User> userList = mapper.findAll();

```

方式1-入口分析

SqlSession是DefaultSqlSession类型的，因为sqlSessionFactory默认生成的SqlSession是DefaultSqlSession类型。selectOne()会调用selectList()。

```

1 DefaultSqlSession#selectOne()
2     Configuration#selectList()//selectList如下
3 // DefaultSqlSession类
4 public <E> List<E> selectList(String statement, Object parameter, RowBounds
rowBounds) {
5     try {
6         MappedStatement ms = configuration.getMappedStatement(statement);
7         // CURD操作是交给Excetor去处理的

```

```

8         return executor.query(ms, wrapCollection(parameter), rowBounds,
Executor.NO_RESULT_HANDLER);
9     } catch (Exception e) {
10         throw ExceptionFactory.wrapException("Error querying database.
Cause: " + e, e);
11     } finally {
12         ErrorContext.instance().reset();
13     }
14 }

```

方式2-入口分析

```

1 DefaultSqlSession#getMapper()
2     Configuration#getMapper()
3         MapperRegistry#getMapper()
4             MapperProxyFactory.newInstance()

```

5.1.2 Mapper接口动态代理原理

代理对象执行逻辑MapperProxy#invoke()

```

1 MapperProxy#invoke()//代理对象执行的方法，代理后所有Mapper的方法调用时，都会调用这个
invoke方法
2     MapperProxy#cachedInvoker()//代理对象里的Method也是对象，为了避免频繁new对象，
在这里给Method对象家缓存
3         methodCache#computeIfAbsent()//从缓存中取Method对象，取不到创建之后加入缓
存
4         //PlainMethodInvoker是MapperProxy的内部类
5         PlainMethodInvoker#invoke()//执行方法，所有代理对象的方法都会执行同一个。代
理对象本质是方法的拦截器！
6         MapperMethod#execute()//执行方法，最终还是在调用SqlSession接口
7             SqlSession#insert()
8             SqlSession#update()
9             SqlSession#delete()
10            SqlSession#selectOne()
11            SqlSession#selectList()

```

5.1.3 查询SQL执行流程

主要步骤：

1. selectOne
2. SQL获取
3. 参数设置
4. SQL执行
5. 结果集封装

```

1 DefaultSqlSession#selectOne()//执行单记录
2 DefaultSqlSession#selectList()//查询列表

```

```

3      CachingExecutor#query()
4          MappedStatement#getBoundSql()//获取SQL语句
5      CachingExecutor#query()//二级缓存查询
6          CachingExecutor.delegate = SimpleExecutor//装饰模式: Caching的对
SimpleExecutor查询加二级缓存装饰
7      SimpleExecutor#query()
8          BaseExecutor#query()//子类执行查询直接调用父类方法, 一级缓存
9          BaseExecutor#queryFromDatabase()//缓存没有则去查询数据库, 一级缓存
10         SimpleExecutor#doQuery()//simple执行查询
11         SimpleExecutor#prepareStatement()//准备查询语句
12         RoutingStatementHandler#parameterize()//路由
delegate=PreparedStatementHandler
13         PreparedStatementHandler#parameterize()//设置查询参数
14         DefaultParameterHandler#setParameters()//设置查询参数
15         RoutingStatementHandler#query()//路由
delegate=PreparedStatementHandler
16         PreparedStatementHandler#query()//执行SQL, 结果集封装
17         PreparedStatement#execute()//执行SQL查询操作
18         DefaultResultSetHandler#handleResultSets()//封装返回值, 将
查询结果封装成Object对象

```

5.1.4 增删改SQL执行流程

主要步骤:

- insert|update|delete方法分析
- SQL获取
- 参数设置
- SQL执行
- 结果集封装

```

1  //DefaultSqlSession
2  @Override
3  public int insert(...) {
4      return update(statement, parameter);
5  }
6  @Override
7  public int update(String statement) {
8      return update(statement, null);
9  }
10 @Override
11 public int delete(...) {
12     return update(...);
13 }

```

```

1  DefaultSqlSession#update()
2      CachingExecutor#update()//执行更新
3      flushCacheIfRequired()//执行增删改前, 清除对应二级缓存
4      CachingExecutor.delegate = SimpleExecutor//装饰模式
5      SimpleExecutor#update()//调用父类模板方法
6      BaseExecutor#update()//执行更新
7      BaseExecutor#clearLocalCache()//清除一级缓存, LocalCache

```

```

8      BaseExecutor#doUpdate()//调用子类方法
9      SimplyExecutor#doUpdate()//simple执行更新
10     SimplyExecutor#prepareStatement()//准备查询语句
11
12     RoutingStatementHandler#parameterize()//delegate=PreparedStatementHandler
13     PreparedStatementHandler#parameterize()//设置查询参数
14     DefaultParameterHandler#setParameters()//设置查询参数
15     RoutingStatementHandler#update()//delegate=PreparedStatementHandler
16     PreparedStatementHandler#update()
17     PreparedStatement#execute()//执行SQL，完成增删改查操作
18     PreparedStatement#getUpdateCount()//获取影响行数

```

5.2 动态SQL实现原理

下面是一段动态SQL语句，请问动态SQL的实现原理是什么？

```

1  <select id="getUserByEntity" resultType="com.hero.pojo.User">
2      select <include refid="userAllField"/> from user
3      <where>
4          <if test="id != null">
5              AND id = #{id}
6          </if>
7          <if test="name!= null">
8              AND name = #{name}
9          </if>
10         <if test="phone != null">
11             AND phone = #{phone}
12         </if>
13     </where>
14 </select>

```

如果想弄清楚动态SQL语句，我们首先的介绍与之相关关系的几个封装类

- SqlSource：封装SQL内容和Mapper参数映射信息
- BoundSql：在SqlSource的基础上，多封装了参数信息
- LanguageDriver：将SQL配置信息转换为SqlSource对象
- SqlNode：MyBatis框架实现动态SQL的基石，描述Mapper SQL配置中的SQL节点

5.2.1 相关组件详解

1) SqlSource与BoundSql

- SqlSource比较简单，只封装了Mapper解析后的SQL内容和Mapper参数映射信息。
- BoundSql是对Executor组件执行SQL完整信息的封装，BoundSql相当于SqlSource再加上参数信息。
- Executor操作数据库使用的是BoundSql，当然不是说SqlSource用不到，BoundSql是通过SqlSource构建得来。

```

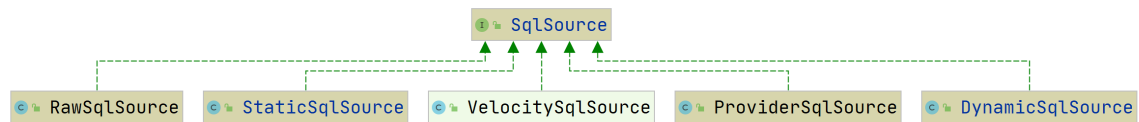
1  public class StaticSqlSource implements SqlSource {
2      // 解析后的SQL
3      private final String sql;

```

```

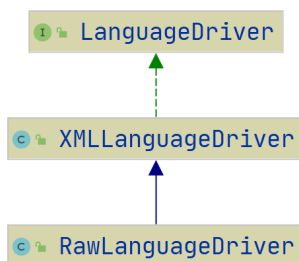
4      // 参数映射信息
5      private final List<ParameterMapping> parameterMappings;
6      private final Configuration configuration;
7      //构造方法省略...
8      @Override
9      public BoundSql getBoundSql(Object parameterObject) {
10         return new BoundSql(configuration, sql, parameterMappings,
11             parameterObject);
12     }
13     }
14     public class BoundSql {
15         private final String sql;//配置解析后的SQL
16         private final List<ParameterMapping> parameterMappings;//参数映射信息
17         private final Object parameterObject;//参数对象
18         private final Map<String, Object> additionalParameters;//额外参数信息
19         private final MetaObject metaParameters;//参数对应的MetaObject对象
20         //省略方法...
21     }

```



2) LanguageDriver

SQL配置信息到SqlSource对象的转换是由LanguageDriver组件来完成的



LanguageDriver接口提供了两个实现类，分别为XMLLanguageDriver和RawLanguageDriver。

- **XMLLanguageDriver**为XML语言驱动，支持动态SQL的功能，通过和标签结合OGNL表达式实现动态SQL
 - createSqlSource(Configuration c, XNode s, Class<?> pt)，解析XML里面的SQL信息
 - createSqlSource(Configuration c, String s, Class<?> pt)，解析注解里面的SQL信息
- **RawLanguageDriver**仅支持静态SQL配置，不支持动态SQL功能，

举个栗子：注解里面可以有动态SQL！

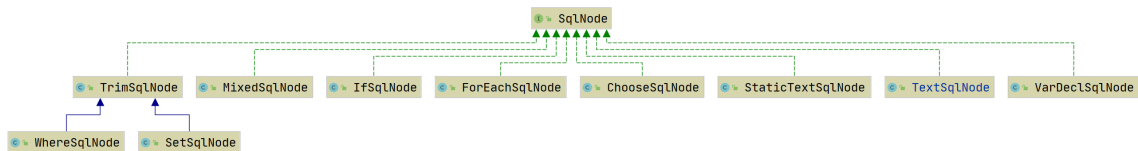
```

1  @Select("<script>"+
2      "select * from user\n"+
3      "<where>\n"+
4      "    <if test=\"name!= null\">\n"+
5      "        AND name =#{name}\n"+
6      "    </if>\n"+
7      "    <if test=\"phone!=null\">\n"+
8      "        AND phone = #{phone}\n" +
9      "    </if>\n"+
10     "</where>"+
11     "</script>")
12 User getUserByPhoneAndName(@Param("phone")String phone ,
    @Param("name")String name);

```

3) SqlNode

SqlNode用于描述Mapper SQL配置中的SQL节点，它是MyBatis框架实现动态SQL的基石



```

1  public interface SqlNode {
2      //用于解析SQL节点，根据参数信息生成静态SQL内容。
3      //DynamicContext对象中封装了Mapper调用时传入的参数信息及MyBatis内置的_parameter和
4      _databaseId参数。
5      boolean apply(DynamicContext context);
6  }

```

动态SQL解析时，会把标签内容转换为对应SqlNode对象，例如转为IfSqlNode

- IfSqlNode：用于描述动态SQL中标签
- ChooseSqlNode：用于描述动态SQL配置中的标签
- ForEachSqlNode：用于描述动态SQL配置中的标签
- SetSqlNode：用于描述动态SQL配置中的标签
- WhereSqlNode：用于描述动态SQL中的标签
- TrimSqlNode：用于描述动态SQL中的标签
- VarDeclSqlNode：用于描述动态SQL中的标签
- StaticTextSqlNode：用于描述动态SQL中的静态文本内容，不含占位符
- TextSqlNode：该类与StaticTextSqlNode类不同的是，SQL包含\${}占位符时，用TextSqlNode描述。
- MixedSqlNode：用于描述一组SqlNode对象，通常一个Mapper配置是由多个SqlNode对象组成的，这些SqlNode对象通过MixedSqlNode进行关联，组成一个完整的动态SQL配置

举个栗子：

```

1 <select id="getUserByWhere" resultType="com.hero.pojo.User">
2     select * from user where 1=1
3     <if test="id != null">
4         AND id = #{id}
5     </if>
6     <if test="name != null">
7         AND name #{name}
8     </if>
9     <if test="phone != null">
10        AND phone #{phone}
11    </if>
12 </select>

```

```

1 public class DynamicSqlExample {
2     @Test
3     public void testSqlNode() throws IOException {
4         SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
5         InputStream inputStream =
6         Resources.getResourceAsStream("SqlMapConfig.xml");
7         SqlSessionFactory sqlSessionFactory = builder.build(inputStream);
8         SqlSession sqlSession = sqlSession.openSession();
9         //构建SqlNode
10        SqlNode sn1 = new StaticTextSqlNode("select * from user where 1=1");
11        SqlNode sn2 = new IfSqlNode(new StaticTextSqlNode("AND id=#{id}"),
12        "id != null");
13        SqlNode sn3 = new IfSqlNode(new StaticTextSqlNode("AND name=#{
14        {name}"), "name != null");
15        SqlNode sn4 = new IfSqlNode(new StaticTextSqlNode(" AND phone=#{
16        {phone}"), "phone != null");
17        SqlNode mixedsqlNode = new MixedSqlNode(Arrays.asList(sn1, sn2, sn3,
18        sn4));
19        //创建参数对象
20        Map<String, Object> paramMap = new HashMap<>();
21        paramMap.put("id", "1");
22        //创建动态SQL解析上下文
23        DynamicContext context = new
24        DynamicContext(sqlSession.getConfiguration(), paramMap);
25        //调用SqlNode的apply()方法解析动态SQL
26        mixedsqlNode.apply(context);
27        //调用DynamicContext对象的getSql()方法获取动态SQL解析后的SQL语句
28        System.out.println(context.getSql());
29    }
30 }

```

5.2.3 动态SQL解析过程

- SqlSource用于描述通过XML文件或者Java注解配置的SQL资源信息;
- SqlNode用于描述动态SQL中、等标签信息;
- LanguageDriver用于对Mapper SQL配置进行解析, 将SQL配置转换为SqlSource对象。

动态SQL解析的入口: XMLLanguageDriver#createSqlSource()方法

```

1 //解析XML里面的动态SQL
2 XMLLanguageDriver#createSqlSource()//
3 //将SQL语句解析为SQLSource，动态SQL则创建DynamicSqlSource，否则创建
  RawSqlSource
4 XMLScriptBuilder#parseScriptNode()
5 XMLScriptBuilder#parseDynamicTags()//将SQL语句，转换为SqlNode对象
6 NodeHandler#handleNode()//把动态SQL标签转换为对应的SqlNode对象，
  NodeHandler接口提供了8个实现类
7 IfHandler#handleNode()//解析if标签
8 whereHandler#handleNode()//解析where标签
9 ChooseHandler#handleNode()//解析choose标签
10 SetHandler#handleNode()//解析set标签
11 ...
12 //当动态SQL解析完成之后，SqlNode封装在SqlSource中，SqlSource创建完成之后会存放在
  MappedStatement中。
13 //当Executor组件操作数据库时，执行getBoundSql时，会将Mapper参数生成动态SQL语句
14 DefaultSqlSession#selectOne()//执行单记录
15 DefaultSqlSession#selectList()//查询列表
16 CachingExecutor#query()
17 MappedStatement#getBoundSql()
18 DynamicSqlSource#getBoundSql()//获取BoundSql对象，用于执行SQL
19 SqlSourceBuilder#parse()//进一步解析SQL
20 GenericTokenParser#parse()//解析#{ }参数占位符
21 ParameterMappingTokenHandler#handleToken()//#{ }占位符内容
  替换为? 占位符
22
  ParameterMappingTokenHandler#buildParameterMapping()//将占位符内容转换为
  ParameterMapping

```

问题：

1. SqlNode对象是如何根据调用Mapper时传入的参数动态生成SQL语句的呢？
2. #{}与\${}的有什么区别？

5.2.3 #{}与\${}的区别

当动态SQL配置中存在\${}参数占位符时，MyBatis会使用TextSqlNode对象描述对应的SQL节点，在调用TextSqlNode对象的apply()方法时会完成动态SQL的解析。

```

1 // ${ }表达式
2 TextSqlNode#apply()//解析动态SQL
3 GenericTokenParser#parse()//解析带$表达式的SQL
4 BindingTokenParser#handleToken()//方法替换参数占位符
5 // #{ }表达式
6 MappedStatement#getBoundSql()
7 DynamicSqlSource#getBoundSql()//获取BoundSql对象，用于执行SQL
8 SqlSourceBuilder#parse()//进一步解析SQL
9 GenericTokenParser#parse()//解析#{ }参数占位符
10 ParameterMappingTokenHandler#handleToken()//#{ }占位符内容替换
  为? 占位符

```

#{}和\${}参数占位符的区别：

- 使用#{ }参数占位符时，占位符内容会被替换成“?”然后通过PreparedStatement对象的setXXX()方法为参数占位符设置值
- 使用\${ }参数占位符时，内容会被直接替换为参数值

知道什么优先考虑使用#了吧?

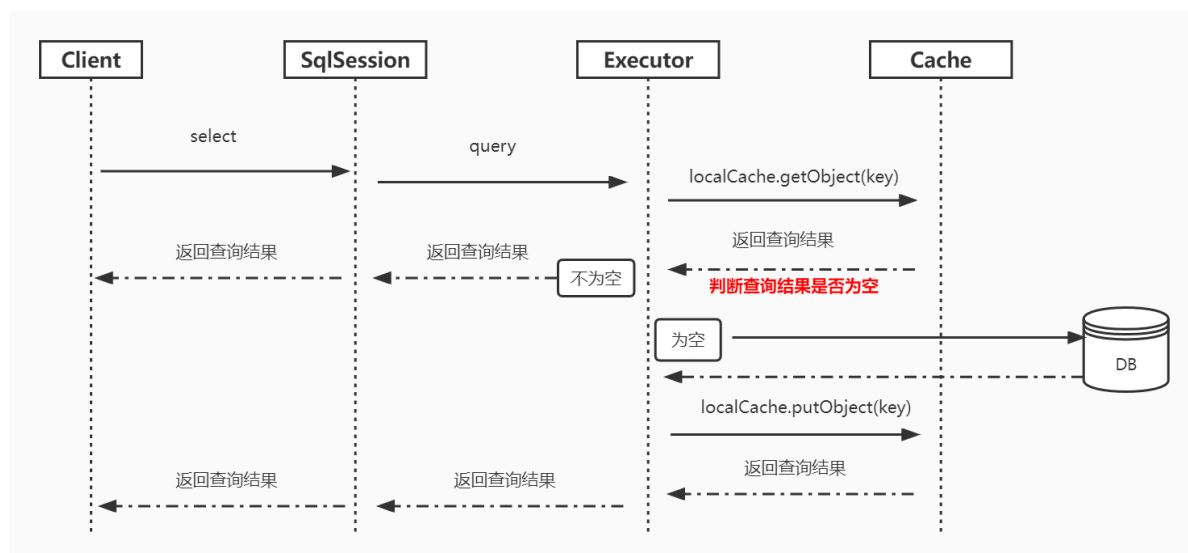
使用#{ }参数占位符能够有效避免SQL注入问题

5.3 缓存原理

5.3.1 一级缓存

一级缓存的生命周期是一个会话的生命周期

- 当会话结束时，SqlSession及Executor、PerpetualCache都会释放掉，一级缓存就清除了
- 当会话没有结束，有两种情况会导致缓存清除
 - SqlSession调用close()方法，释放PerpetualCache对象，一级缓存将不可用
 - SqlSession调用clearCache()，清空PerpetualCache对象的数据，一级缓存还可使用



```

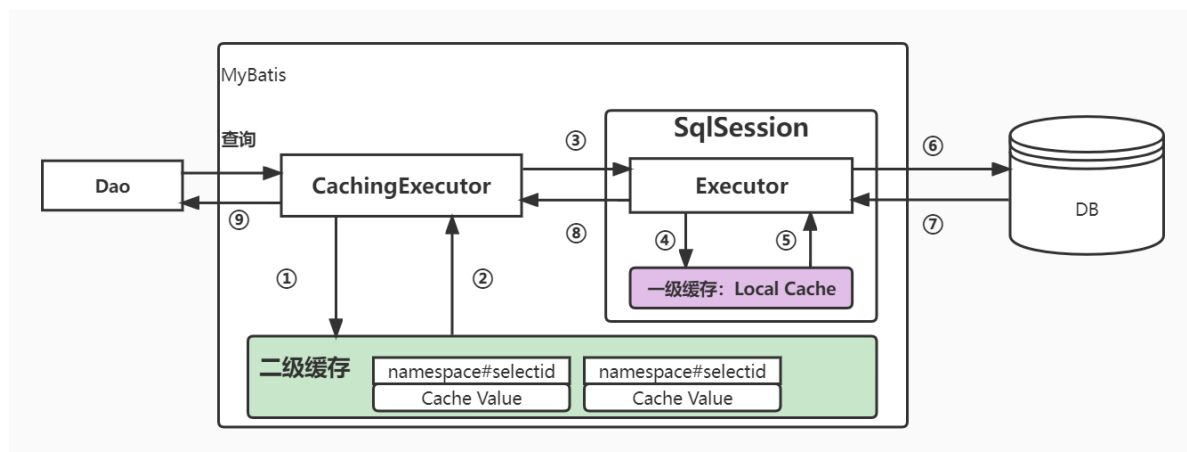
1 DefaultSqlSession#selectOne()//执行单记录
2 DefaultSqlSession#selectList()//查询列表
3   CachingExecutor#query()
4     MappedStatement#getBoundSql()//获取SQL语句
5   CachingExecutor#query()//二级缓存查询
6     CachingExecutor.delegate = SimpleExecutor//装饰模式: Caching的对
SimpleExecutor查询加二级缓存装饰
7   SimpleExecutor#query()
8     BaseExecutor#query()//子类执行查询直接调用父类方法，一级缓存
9     BaseExecutor#queryFromDatabase()//缓存没有则去查询数据库，一级缓存
  
```

5.3.2 二级缓存

一级缓存的生命周期是一个会话的生命周期，而二级缓存的生命周期是整个SessionFactory，所有Session共享一个缓存区域。

- 二级缓存需手动开启，开启后执行器Executor会被CachingExecutor包装一层。
- 只有在提交事务之后，才会新增二级缓存项
- 查询时，先走二级缓存，查不到再走一级缓存

装饰者模式：在不改变原有类和继承的情况下，通过 包装原有类 去扩展新功能。



org.apache.ibatis.executor.CachingExecutor，实现 Executor 接口，支持二级缓存的 Executor的实现类。

- 先从缓存中获取查询结果，存在就返回，不存在，再委托给Executor delegate去数据库取
- delegate可以是SimpleExecutor、ReuseExecutor、BatchExecutor。
- 获得 Cache 对象，即当前 MappedStatement 对象的二级缓存
- 如果没有 Cache 对象，说明该 MappedStatement 对象，未设置二级缓存，则调用delegate 属性的 #query(...) 方法，直接从数据库中查询。
- 如果有 Cache 对象，说明该 MappedStatement 对象，有设置二级缓存：

6. 插件原理与应用

MyBatis插件本质上其实是一个拦截器InterceptorChain，用户自定义的拦截器也被称为MyBatis插件。

MyBatis框架允许用户通过自定义拦截器的方式改变SQL的执行行为。

例如：在SQL执行时追加SQL分页语法，从而达到简化分页查询的目的。

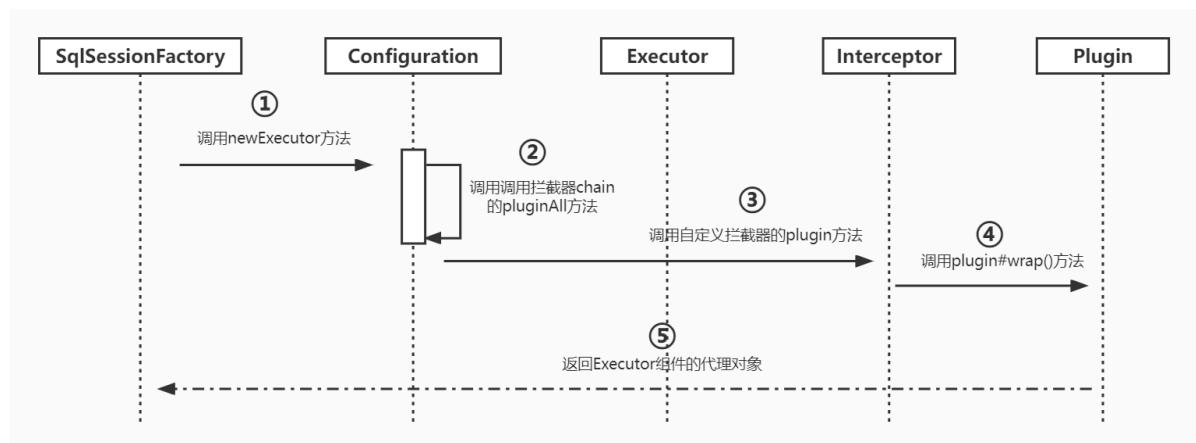
6.1 MyBatis插件实现原理

```
1 <plugins>
2   <plugin interceptor="com.hero.plugin.pager.PageInterceptor">
3     <!-- 支持MySQL和Oracle -->
4     <property name="databaseType" value="mysql"/>
5   </plugin>
6 </plugins>
```

以执行一个查询操作为例，介绍一个MyBatis插件实现原理

SqlSession获取Executor实例的过程如下：

1. SqlSessionFactory中会调用Configuration类提供的newExecutor()工厂方法创建Executor对象
2. Configuration类中通过一个InterceptorChain对象维护了用户自定义的拦截器链。
3. newExecutor()工厂方法中调用InterceptorChain对象的pluginAll()方法
4. InterceptorChain对象的pluginAll()方法中会调用自定义拦截器的plugin()方法
5. 自定义拦截器的plugin()方法是由我们来编写的，通常会调用Plugin类的wrap()静态方法创建一个代理对象

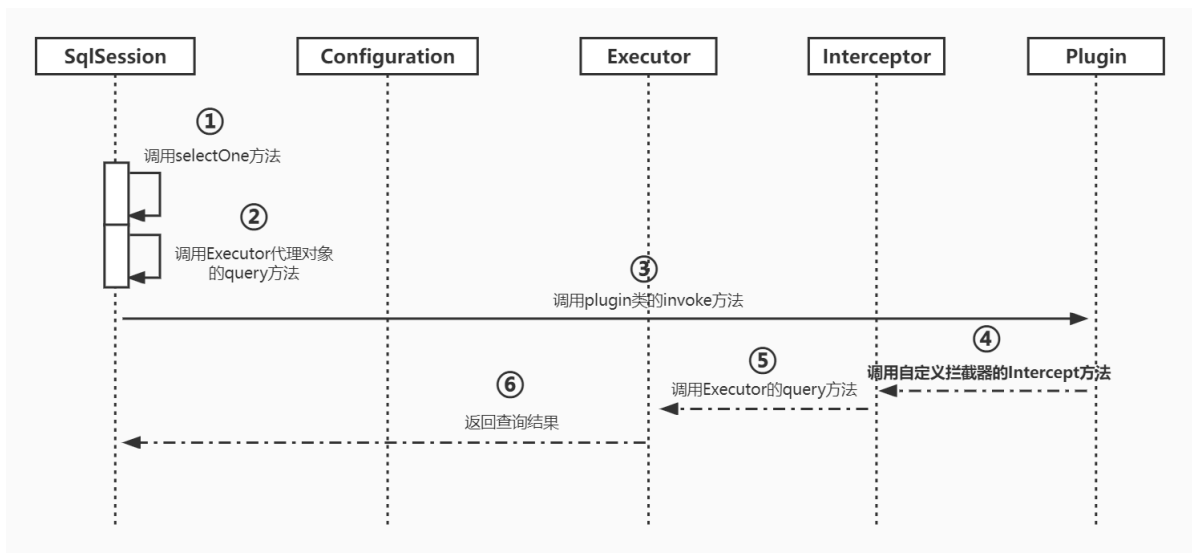


```
1 DefaultSqlSessionFactory#openSessionFromDataSource()//开启会话
2 Configuration.newExecutor()//创建执行器
3 executor = new CachingExecutor(executor);////创建缓存类型的执行器，装饰器
模式
4 InterceptorChain.pluginAll();//为执行器设置拦截器链
5 //当Executor执行的时候，就会触发拦截器，这其中包含我们编写的自定义拦截器
```

SqlSession获取到的Executor实例实际上是个动态代理对象。

接下来，我们就以SqlSession执行查询操作为例，介绍自定义插件执行拦截逻辑的过程。

1. SqlSession操作数据库依赖于Executor，SqlSession会调用Configuration#newExecutor()方法获取Executor代理对象
2. SqlSession获取的是Executor组件的代理对象，执行查询操作时，会调用代理对象的query()方法
3. 按照JDK动态代理机制，调用Executor代理对象的query()方法时，会调用Plugin类的invoke()方法
4. Plugin类的invoke()方法中会调用自定义拦截器对象的intercept()方法执行拦截逻辑
5. 自定义拦截器对象的intercept()方法调用完毕后，调用目标Executor对象的query()方法
6. 所有操作执行完毕后，会将查询结果返回给SqlSession对象



```

1 SqlSessionFactory#selectOne()//执行查询
2 SqlSessionFactory#selectList()//执行查询
3   ExecutorProxy#query()//代理执行器执行查询
4     Plugin#invoke();//插件拦截
5       MyInterceptor#intercept()//调用自定义拦截器的拦截方法
6         Executor#query()//执行被代理的Executor的query方法

```

6.2 实战案例：自定义一个分页插件

6.2.1 设计分页插件

1) PageInterceptor拦截器

```

1 // MyBatis用户自定义插件类都必须实现Interceptor接口，因此我们自定义的PageInterceptor
  类也实现了该接口
2 @Intercepts({
3     // 这里指定对StatementHandler实例的prepare()方法进行拦截，
4     @Signature(method = "prepare", type = StatementHandler.class, args =
5     {Connection.class, Integer.class})
6 })
7 public class PageInterceptor implements Interceptor {
8     private String databaseType;
9
10    // 我们只需要在该方法中把执行的SQL语句替换成分页查询SQL即可
11    public Object intercept(Invocation invocation) throws Throwable {
12        // 获取拦截的目标对象
13        RoutingStatementHandler handler = (RoutingStatementHandler)
14        invocation.getTarget();
15        StatementHandler delegate = (StatementHandler)
16        ReflectionUtils.getFieldValue(handler, "delegate");
17        BoundSql boundSql = delegate.getBoundSql();
18        // 获取参数对象，当参数对象为Page的子类时执行分页操作
19        Object parameterObject = boundSql.getParameterObject();
20        if (parameterObject instanceof Page<?>) {
21            Page<?> page = (Page<?>) parameterObject;
22        }
23    }
24 }

```

```

20         MappedStatement mappedStatement = (MappedStatement)
ReflectionUtils.getFieldValue(delegate, "mappedStatement");
21         Connection connection = (Connection) invocation.getArgs()[0];
22         String sql = boundSql.getSql();
23         if (page.isFull()) {
24             // 获取记录总数
25             this.setTotalCount(page, mappedStatement, connection);
26         }
27         page.setTimestamp(System.currentTimeMillis());
28         // 获取分页SQL
29         String pageSql = this.getPageSql(page, sql);
30         // 将原始SQL语句替换成分页语句
31         ReflectionUtils.setFieldValue(boundSql, "sql", pageSql);
32     }
33     return invocation.proceed();
34 }
35 /**
36  * 根据page对象获取对应的分页查询Sql语句,
37  * 这里只做了三种数据库类型, MySQL、Oracle
38  * 其它的数据库都没有进行分页
39  * @param page 分页对象
40  * @param sql 原始sql语句
41  */
42 private String getPageSql(Page<?> page, String sql) {
43     StringBuffer sqlBuffer = new StringBuffer(sql);
44     if ("mysql".equalsIgnoreCase(databaseType)) {
45         return getMysqlPageSql(page, sqlBuffer);
46     } else if ("oracle".equalsIgnoreCase(databaseType)) {
47         return getOraclePageSql(page, sqlBuffer);
48     }
49     return sqlBuffer.toString();
50 }
51 /**
52  * 获取Mysql数据库的分页查询语句
53  *
54  * @param page 分页对象
55  * @param sqlBuffer 包含原sql语句的StringBuffer对象
56  * @return Mysql数据库分页语句
57  */
58 private String getMysqlPageSql(Page<?> page, StringBuffer sqlBuffer) {
59     int offset = (page.getPageNo() - 1) * page.getPageSize();
60     sqlBuffer.append(" limit
").append(offset).append(", ").append(page.getPageSize());
61     return sqlBuffer.toString();
62 }
63 //...省略不重要代码
64 }

```

2) Page类

```

1 public class Page<T> implements Pageable<T> {
2     public static final int DEFAULT_PAGE_SIZE = 10; // 默认每页记录数
3     public static final int PAGE_COUNT = 10;
4     private int pageNo = 1; // 页码
5     private int pageSize = DEFAULT_PAGE_SIZE; // 每页记录数
6     private int totalCount = 0; // 总记录数
7     private int totalPage = 0; // 总页数
8     //...
9 }

```

3) Pageable接口

```

1 public interface Pageable<T> {
2     /** 总记录数 */
3     int getTotalCount();
4     /** 总页数 */
5     int getTotalPage();
6     /** 每页记录数 */
7     int getPageSize();
8     /** 当前页号 */
9     int getPageNo();
10    /** 是否第一页 */
11    boolean isFirstPage();
12    /** 是否最后一页 */
13    boolean isLastPage();
14    /** 返回下页的页号 */
15    int getNextPage();
16    /** 返回上页的页号 */
17    int getPrePage();
18    /** 取得当前页显示的项的起始序号 */
19    int getBeginIndex();
20    /** 取得当前页显示的末项序号 */
21    int getEndIndex();
22    /** 获取开始页 */
23    int getBeginPage();
24    /** 获取结束页 */
25    int getEndPage();
26 }

```

6.2.2 使用分页插件

1) 注册插件

在Mybatis主配置文件中注册插件

```

1 <plugins>
2     <plugin interceptor="com.hero.plugin.pager.PageInterceptor">
3         <!-- 支持MySQL和Oracle -->
4         <property name="databaseType" value="mysql"/>
5     </plugin>
6 </plugins>

```

2) Mapper接口使用分页插件

定义Mapper接口，使用Page的子类作为参数，使用分页查询数据

```
1 public interface UserMapper {
2     @Select("select * from t_user")
3     List<User> getUserPageable(HeroPager pager);
4
5     @Insert("insert into t_user (id,name,age,address) values (#{id},#{
6         {name},{age},{address}});")    void save(User user);
7
8     @Delete("delete from t_user")
9     void deleteAll();
10 }
11
12 public class HeroPager extends Page<User> {}
```

3) 测试分页查询代码

```
1 public class HeroPagerExample {
2
3     private UserMapper userMapper;
4
5     private SqlSession sqlSession;
6
7     @Before
8     public void init() throws IOException {
9         // 获取配置文件输入流
10        InputStream inputStream = Resources.getResourceAsStream("mybatis-
11        config.xml");
12        // 通过SqlSessionFactoryBuilder的build()方法创建SqlSessionFactory实例
13        SqlSessionFactory sqlSessionFactory = new
14        SqlSessionFactoryBuilder().build(inputStream);
15        // 调用openSession()方法创建SqlSession实例
16        sqlSession = sqlSessionFactory.openSession();
17        // 获取UserMapper代理对象
18        userMapper = sqlSession.getMapper(UserMapper.class);
19        //初始化一些数据
20        initUsers(userMapper);
21    }
22
23    private void initUsers(UserMapper userMapper) {
24        userMapper.deleteAll();
25
26        for (int i = 1; i < 20; i++) {
27            User user = User.builder().id(Long.valueOf(i)).name("刘备【" + i
28            + "】").age(11).address("蜀国").build();
29            userMapper.save(user);
30        }
31        sqlSession.commit();
32    }
33
34    @Test
35    public void testPageInterceptor() {
36        HeroPager pager = new HeroPager();
37        pager.setPageSize(5);
38    }
39}
```

```
35     pager.setFull(true);
36     List<User> users = userMapper.getUserPageable(pager);
37     System.out.println("总数据量: " + pager.getTotalCount() + ",总页数: " +
pager.getTotalPage()+ ", 当前查询数据: " + JSON.toJSONString(users));
38     sqlSession.close();
39 }
40
41 }
```

测试查询结果: